

Ex. 2.4 (5 pts) — Why a thread context switch is much cheaper than a process one

The claim holds for two threads *of the same process*. Threads belonging to different processes cost exactly what a process switch costs, because the expense comes from changing the address space, not from changing the scheduling entity.

Threads of one process share an address space, so the switch leaves the memory map untouched: the kernel never reloads the page-table base register (**CR3** on x86, **TTBR** on ARM) and so never invalidates the TLB. A process switch does both, and the register saving is the smaller half of its bill. The larger half is indirect and is paid after the switch returns: the incoming process resumes with a cold TLB and with caches still holding the outgoing process's lines, and stalls while both refill. Tagged TLBs (ARM ASIDs, x86 PCIDs) let entries for several address spaces coexist and soften this, but do not remove it, since a different working set still displaces cached data.

A thread switch still saves and restores the small per-thread state: the register file, including the program counter and stack pointer, and the thread's kernel stack and control block. Per-process state such as the open-file table and signal dispositions is shared, so none of it is swapped. A switch between user-level threads avoids even the trap into the kernel.